Unsupervised Learning: Self-organizing Maps

Video Lesson

Duration: 00:45:35

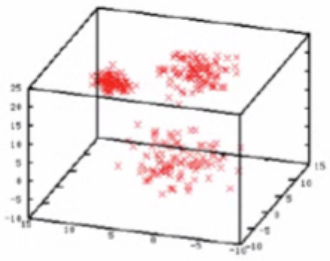
Slide Deck

M12S06 (https://canvas.tamu.edu/courses/430127/files/92570699?wrap=1)

Lesson Transcript

In this lecture, we are looking into unsupervised learning. Most specifically, we will look at self-organizing maps.

Unsupervised Learning



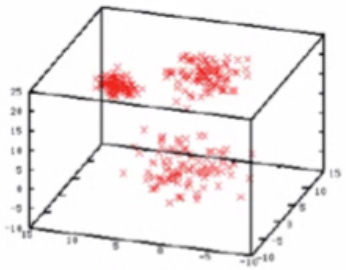
- No teacher signal (i.e. no feedback from the environment).
- The network must discover patterns, features, regularities, correlations, or categories in the input data and code them in the output.
- The units and connections must display some degree of **self-organization**.
- Unsupervised learning can be useful when there is **redundancy** in the input data.
- A data channel where the input data content is less than the channel capacity, there is redundancy.

In unsupervised learning, we are usually given a bunch of input points in the dataset, so there is no teacher signal, class, label, or any kind of feedback.

Machine learning models must discover patterns, features, regularities, correlations, or categories in the input data and code them in the output. If we have a neural network, the units and connections must display some degree of self-organization.

Unsupervised learning can be useful when there is redundancy in the data. That means there is some kind of structure in the input. One other way of looking at redundancy is by comparing the channel capacity with the information content. The data channel, where the input data content is less than the channel capacity. We say that there is redundancy, and this kind of data is suitable for unsupervised learning.

What Can Unsupervised Learning Do?



- **Familiarity**: how similar is the current input to past inputs?
- **Principal Component Analysis**: find orthogonal basis vectors (or axes) against which to project high dimensional data.
- **Clustering**: *n* output class, each representing a distinct category. Each cluster of similar or nearby patterns will be classified as a single class.
- **Prototyping**: For a given input, the most similar output class (or **exemplar**) is determined.
- **Encoding**: application of clustering/prototyping.
- **Feature Mapping**: topographic mapping of input space onto output network configuration.

What can we do with unsupervised learning? There are six different types of tasks: familiarity, principal component analysis, clustering, prototyping, encoding, and feature mapping. Let us look at these ones by one.

First, familiarity. Given a current input that did not appear in the regional dataset, we want to know how similar this current input is to the past inputs that are in the dataset.

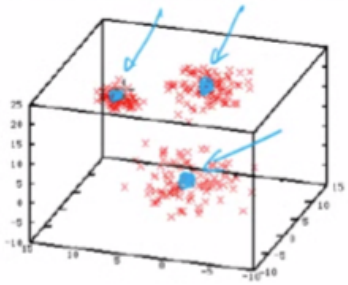
Next, principal component analysis. This is a very widely known technique in machine learning in data science. Thus, in this case, we want to find the orthogonal basis factors or the axes against which to project the high-dimensional data. So, this achieves dimensionality reduction.

Next, clustering. So, the data session here is a good example of this clustering task. We are given a bunch of input points and no labeling, but we can see that there are perhaps three different subsets or subgroups, and these are called clusters. Each of these subsets might represent some kind of distinct category, although we do not know what the category name is for these different clusters that have been found.

Each cluster of similar or nearby patterns will be classified as a single class.

Next is prototyping. For a given input, what is the most similar output class or the exemplar? We want to determine this.

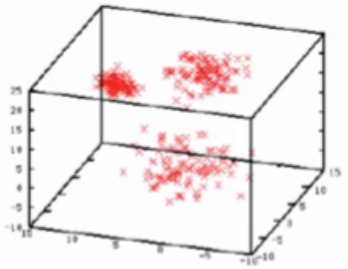
What Can Unsupervised Learning Do?



- **Familiarity**: how similar is the current input to past inputs?
- **Principal Component Analysis**: find orthogonal basis vectors (or axes) against which to project high dimensional data.
- **Clustering**: *n* output class, each representing a distinct category. Each cluster of similar or nearby patterns will be classified as a single class.
- **Prototyping**: For a given input, the most similar output class (or **exemplar**) is determined.
- **Encoding**: application of clustering/prototyping.
- **Feature Mapping**: topographic mapping of input space onto output network configuration.

As shown in this illustration, the blue points at the center of each cluster may be the prototype or the exemplar.

What Can Unsupervised Learning Do?



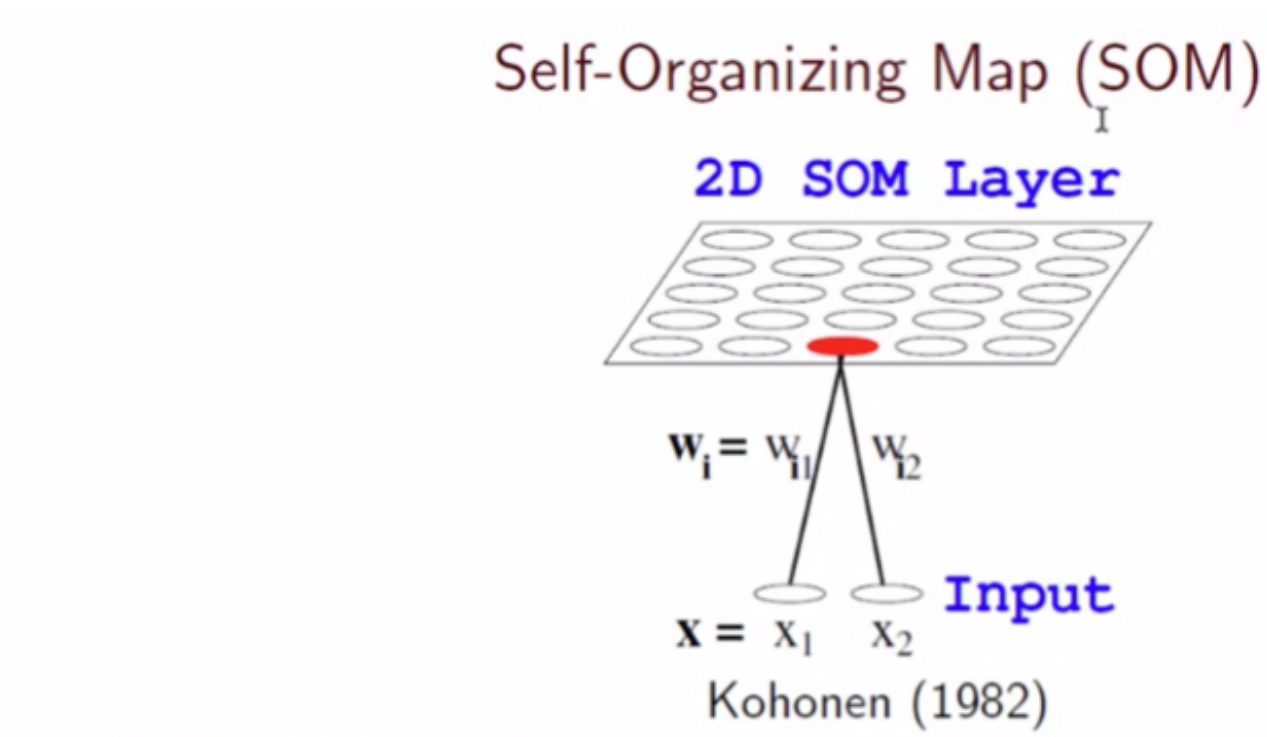
- **Familiarity**: how similar is the current input to past inputs?
- **Principal Component Analysis**: find orthogonal basis vectors (or axes) against which to project high dimensional data.
- **Clustering**: *n* output class, each representing a distinct category. Each cluster of similar or nearby patterns will be classified as a single class.
- **Prototyping**: For a given input, the most similar output class (or **exemplar**) is determined.
- **Encoding**: application of clustering/prototyping.
- **Feature Mapping**: topographic mapping of input space onto output network configuration.

Combining clustering and prototyping, we can also do the encoding. For example, given an arbitrary point, we can encode that as one of these prototype vectors or exemplars.

Finally, there is feature mapping. Here we want to find the topographic mapping of input space

onto the output network configuration.

Up to this point, we just considered the features, but in this case, we also want to find the spatial relationship between the features themselves.



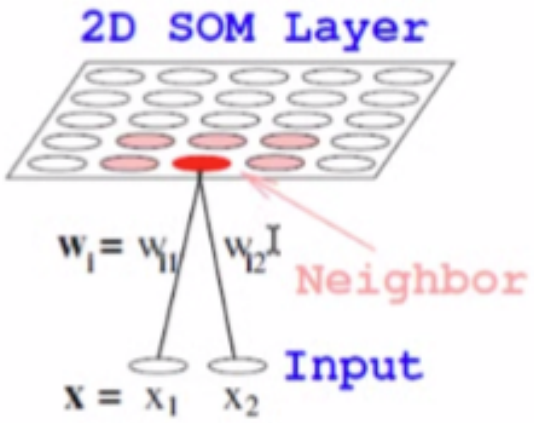
- 1-D, 2-D, or 3-D layout of units.
- One reference vector for each unit.
- Unsupervised learning (no target output).

The self-organizing map is one instance of the feature map. This is commonly abbreviated as SOM or S.O.M.

In S.O.M., we have these three different types of arrangements. We can have either a 1-D chain, a 2-D grid like this, or a 3-D grid as the output layer. And for each unit in the grid, we have one reference vector. This reference vector has the same dimension as the input vector. If we have two values in the input, then we have two values in the weight vector.

The goal of S.O.M. is to learn these weight vectors so that they represent the inputs well, especially the inputs that appear in the training set. Also, make sure that these neighboring units in the grid have similar weight vectors.

SOM Algorithm



1. Randomly initialize reference vectors $\mathbf{w}_j$
2. Randomly sample input vector $\mathbf{x}$
3. Find Best Matching Unit (BMU):

$$i(\mathbf{x}) = \text{argmin}_j \parallel \mathbf{x} - \mathbf{w}_j \parallel$$

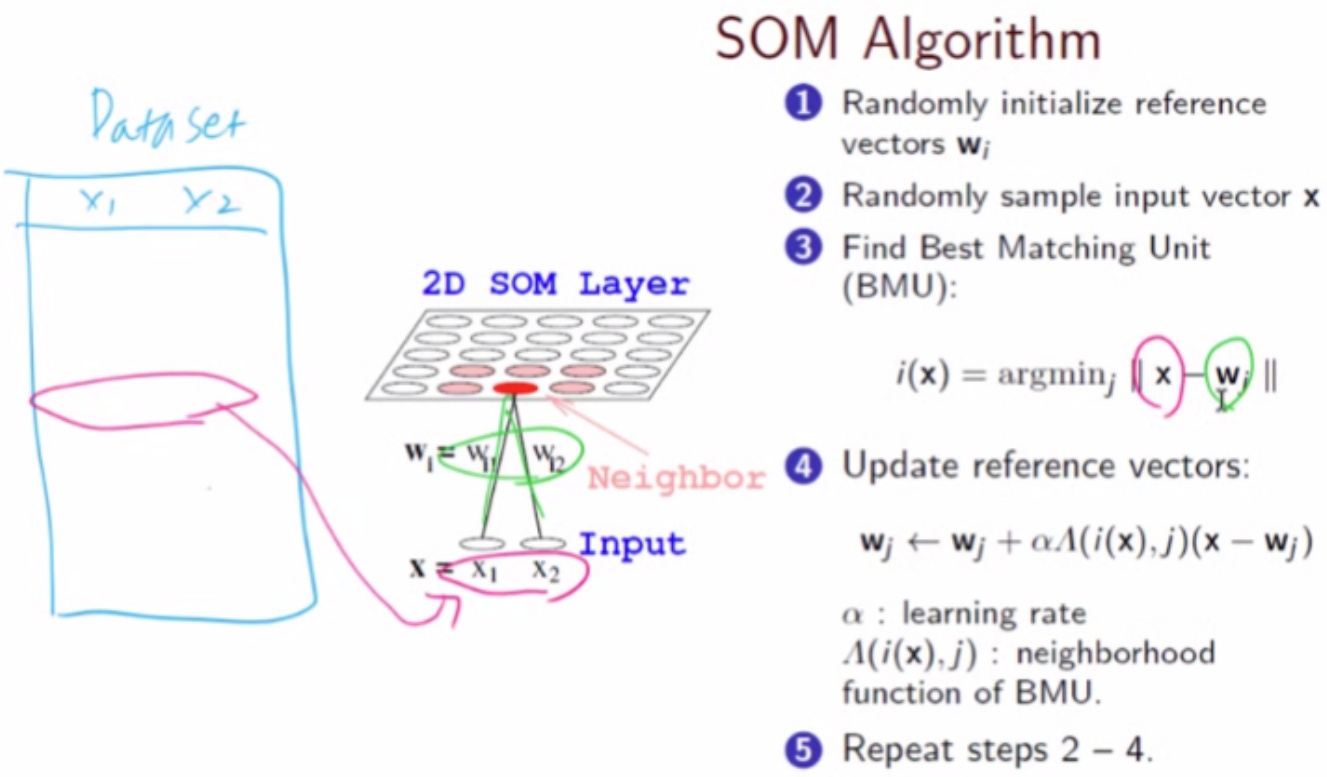
4. Update reference vectors:

$$\mathbf{w}_j \leftarrow \mathbf{w}_j + \alpha \Lambda(i(\mathbf{x}), j)(\mathbf{x} - \mathbf{w}_j)$$

α : learning rate
$\Lambda(i(\mathbf{x}), j)$: neighborhood function of BMU.

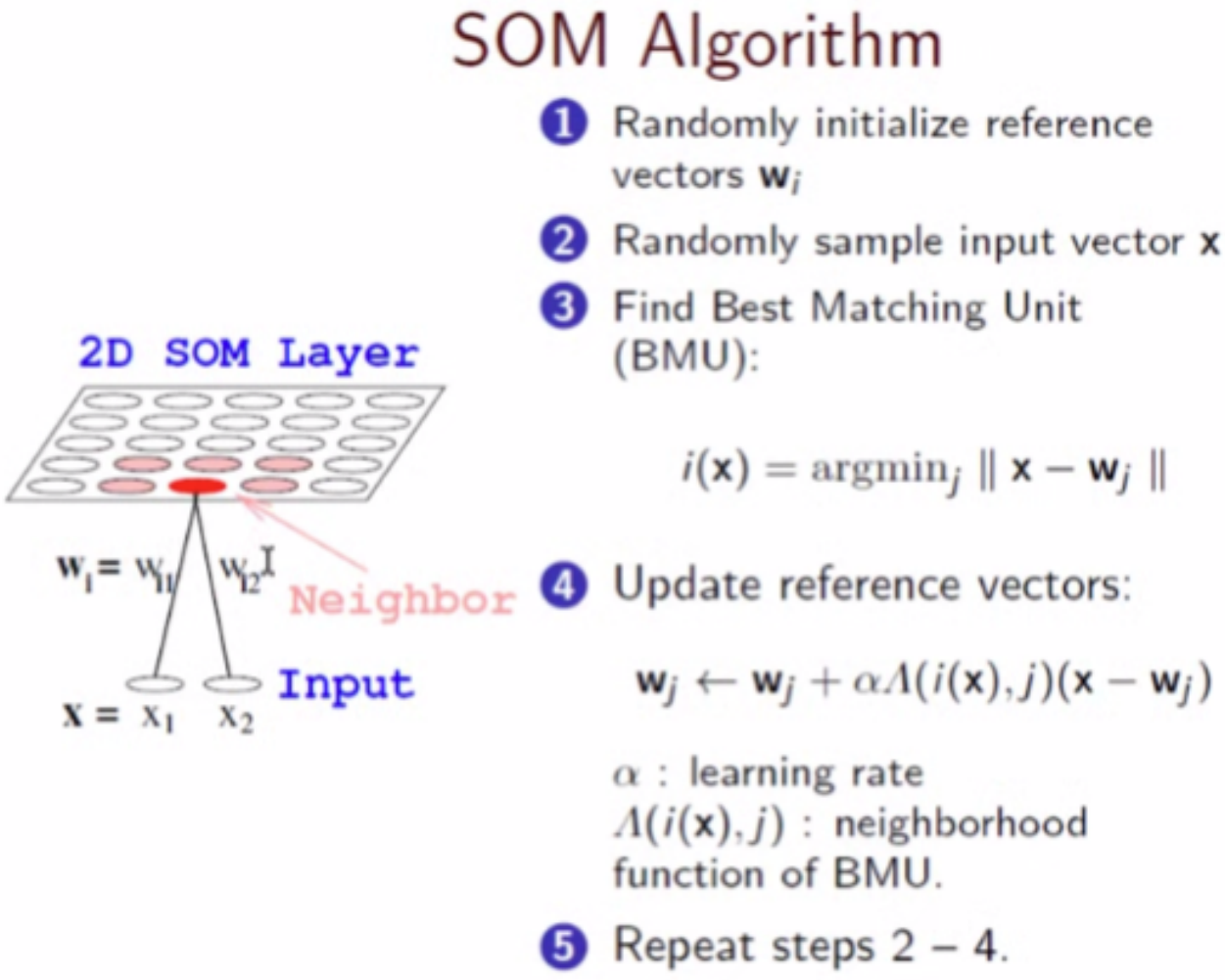
5. Repeat steps 2 – 4.

Here is the S.O.M. algorithm. First, we randomly initialize all the reference vectors, w_j, in the grid. So, each unit here in the grid will have a weight vector. So, we assign random values to w_{i1} and w_{i2} for an arbitrary i.



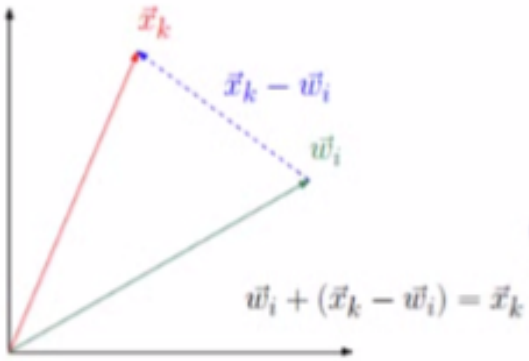
Next, we randomly sample an input vector from the dataset and then go through all of these weight vectors in the grid and find the best matching unit. For example, this unit has a weight vector that is the most similar to the input that we have just picked. Then that becomes the best matching unit.

The units in the S.O.M. layer are indexed by *j*. *j* equals 1, *j* equals 2, *j* equals 3, and so on. And this function *i*(*x*). We plug in the input vector *x*. It will give us the index for the weight matrix, the weight vector of the unit, that is the closest in terms of the Euclid distance. We compare the input vector *x* and the weight vector of unit *j*, and if it is the smallest, then this *j* corresponding to that particular weight vector will be the index of the best matching unit.



Next, we update all of the weights in the grid. So, this update rule is applied to all *j*. Here we have 5 × 5 units. This *j* will go over this updated equation 25 times. The current weight vector is updated as the previous weight vector plus α, the learning rate, multiplied by something called the neighborhood function.

So, this neighborhood function value depends on what was the index for the best matching unit. And what is our current location in the grid indicated by *j*? Finally, multiply by the difference between the input vector and that specific weight vector for unit *j*. So, this will be a vector, and these two values would be scalar values.



SOM Algorithm

1. Randomly initialize reference vectors $\mathbf{w}_j$
2. Randomly sample input vector $\mathbf{x}$
3. Find Best Matching Unit (BMU):

$$i(\mathbf{x}) = \text{argmin}_j \parallel \mathbf{x} - \mathbf{w}_j \parallel$$

4. Update reference vectors:

$$\mathbf{w}_j \leftarrow \mathbf{w}_j + \alpha \Lambda(i(\mathbf{x}), j)(\mathbf{x} - \mathbf{w}_j)$$

α : learning rate
$\Lambda(i(\mathbf{x}), j)$: neighborhood function of BMU.

5. Repeat steps 2 – 4.

Let us consider this updated equation. So, w_j is updated as w_j plus α times the scalar value, which is the neighborhood function value times x minus w_j. If both α and the neighborhood function value were 1, then this right-hand side reduces to w_i, w_j in this case plus x_k minus w_j. So, w_j plus x minus w_j. And because we have w_j and minus w_j, these two cancel out. Hence, the right-hand side would be just x.

That means when we have a learning rate of one and the neighborhood function value of one, then this weight update will move the weight vector exactly to the input vector's location. As we can see, x minus w was the updated amount.

SOM Algorithm

1. Randomly initialize reference vectors $\mathbf{w}_i$
2. Randomly sample input vector $\mathbf{x}$
3. Find Best Matching Unit (BMU):

$$i(\mathbf{x}) = \text{argmin}_j \parallel \mathbf{x} - \mathbf{w}_j \parallel$$

4. Update reference vectors:

$$\mathbf{w}_j \leftarrow \mathbf{w}_j + \alpha\Lambda(i(\mathbf{x}), j)(\mathbf{x} - \mathbf{w}_j)$$

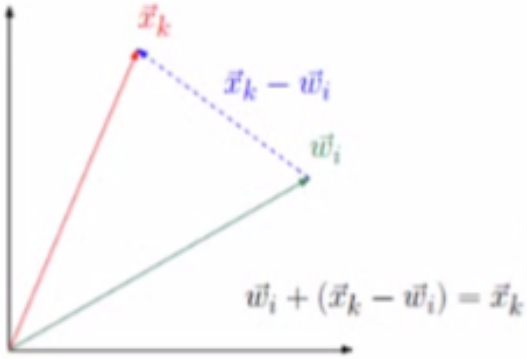
α : learning rate
$\Lambda(i(\mathbf{x}), j)$: neighborhood function of BMU.

5. Repeat steps 2 – 4.

If this combined value were 0.5, then the updated amount here would be 0.5 times *x* minus *w*. Because that is half of this full vector, the update would be this much.

As a result, this vector will move to this location by adding this vector.

SOM Algorithm



1. Randomly initialize reference vectors $\mathbf{w}_i$
2. Randomly sample input vector $\mathbf{x}$
3. Find Best Matching Unit (BMU):

$$i(\mathbf{x}) = \text{argmin}_j \parallel \mathbf{x} - \mathbf{w}_j \parallel$$

4. Update reference vectors:

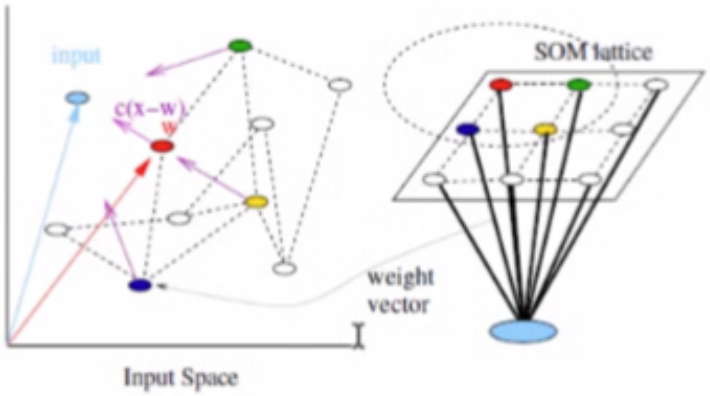
$$\mathbf{w}_j \leftarrow \mathbf{w}_j + \alpha \Lambda(i(\mathbf{x}), j)(\mathbf{x} - \mathbf{w}_j)$$

α : learning rate
$\Lambda(i(\mathbf{x}), j)$: neighborhood function of BMU.

5. Repeat steps 2 – 4.

Finally, we repeat steps two to four. We sample another input. Find the best matching unit, and then update the reference vector, all of it, using the neighborhood function, which is dependent on which one was the best matching unit, with that input *x*.

SOM Algorithm



1. Randomly initialize reference vectors $\mathbf{w}_j$
2. Randomly sample input vector $\mathbf{x}$
3. Find Best Matching Unit (BMU):

$$i(\mathbf{x}) = \text{argmin}_j \parallel \mathbf{x} - \mathbf{w}_j \parallel$$

4. Update reference vectors:

$$\mathbf{w}_j \leftarrow \mathbf{w}_j + \alpha \Lambda(i(\mathbf{x}), j)(\mathbf{x} - \mathbf{w}_j)$$

α : learning rate
$\Lambda(i(\mathbf{x}), j)$: neighborhood function of BMU.

5. Repeat steps 2 – 4.

Here is an illustration of how this process works.

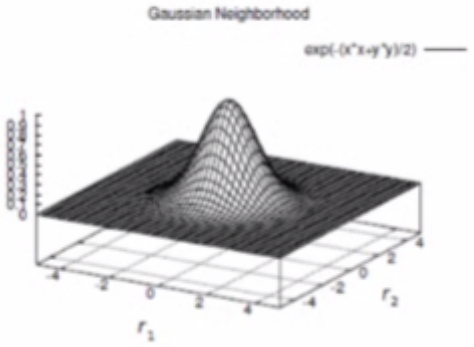
Let us say we have 3 × 3 units in the 2-D grid, and we plotted the weight vector for each unit in the input space. Because these have two values in them, the input has two values. We can plot the weight vector directly in the input space and link them up according to their relation on the grid. Thus, the red one here is connected directly to the green one, and also, this red one here is connected directly to the blue one. Again, we plot the weight vector here in the input space.

Now consider a sampled input, and let us say the location was here. Then we look at all of these weight vectors and find out which one is the closest to the input. And in this case, this red unit would be the best matching unit.

The main idea for this λ function is that this λ has a high value when the unit is near the best matching unit. So, this unit will have the highest λ, and this one will have slightly lower, and so on, and these would have very low values.

Because that value is multiplied as a factor for this, what will happen is that this unit here, its weight vector, will move the closest to the input, and then these would also move proportionally. Whereas these other ones are far away from the best matching unit, and they will not move toward the input.

Typical Neighborhood Functions

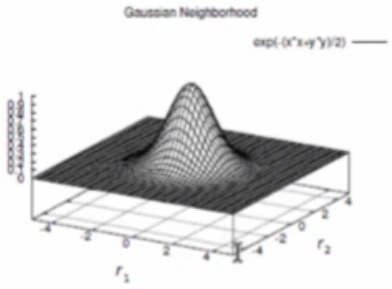


- Gaussian: $\Lambda(i(\mathbf{x}), j) = exp(-|\mathbf{r}_{j\mathbb{I}} - \mathbf{r}_{i(\mathbf{x})}|^2/2\sigma^2)$
  where $\mathbf{r}_j$ is the coordinate of unit j on the grid.
- Flat: $\Lambda(i(x), j) = 1$ if $|\mathbf{r}_j - \mathbf{r}_{i(\mathbf{x})}| \leq \sigma$, and 0 otherwise.
- σ is called the **neighborhood radius**.

Here is a typical neighborhood function that has this 2-dimensional Gaussian shape. And this is for the 2-D grid. We take two arguments, the best matching units index and then the particular unit that we are interested in.

If this *j* is the same as that best matching unit's index, then the neighborhood function value would be the highest given this particular form, which is a Gaussian function. Exponential of minus the distance between the coordinate of unit *j* and the coordinate of the best matching unit on the grid. So, we can see that this is r_1, r_2.

Typical Neighborhood Functions



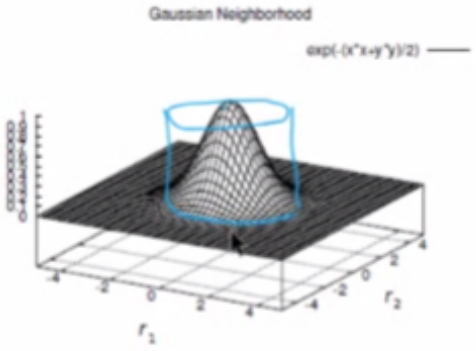
$\vec{r} = (r_1, r_2)$

- Gaussian: $\Lambda(i(\mathbf{x}), j) = exp(-|\mathbf{r}_j - \mathbf{r}_{i(\mathbf{x})}|^2/2\sigma^2)$
  where $\mathbf{r}_j$ is the coordinate of unit j on the grid.
- Flat: $\Lambda(i(x), j) = 1$ if $|\mathbf{r}_j - \mathbf{r}_{i(\mathbf{x})}| \leq \sigma$, and 0 otherwise.
- σ is called the **neighborhood radius**.

So, that vector here would have two components, which are r_1 and r_2, indicating the location on the grid. The σ here is the standard deviation. If we have a high value, then we will have a very spread-out shape like that. If we have a small value, then we will have a very peaked shape like that.

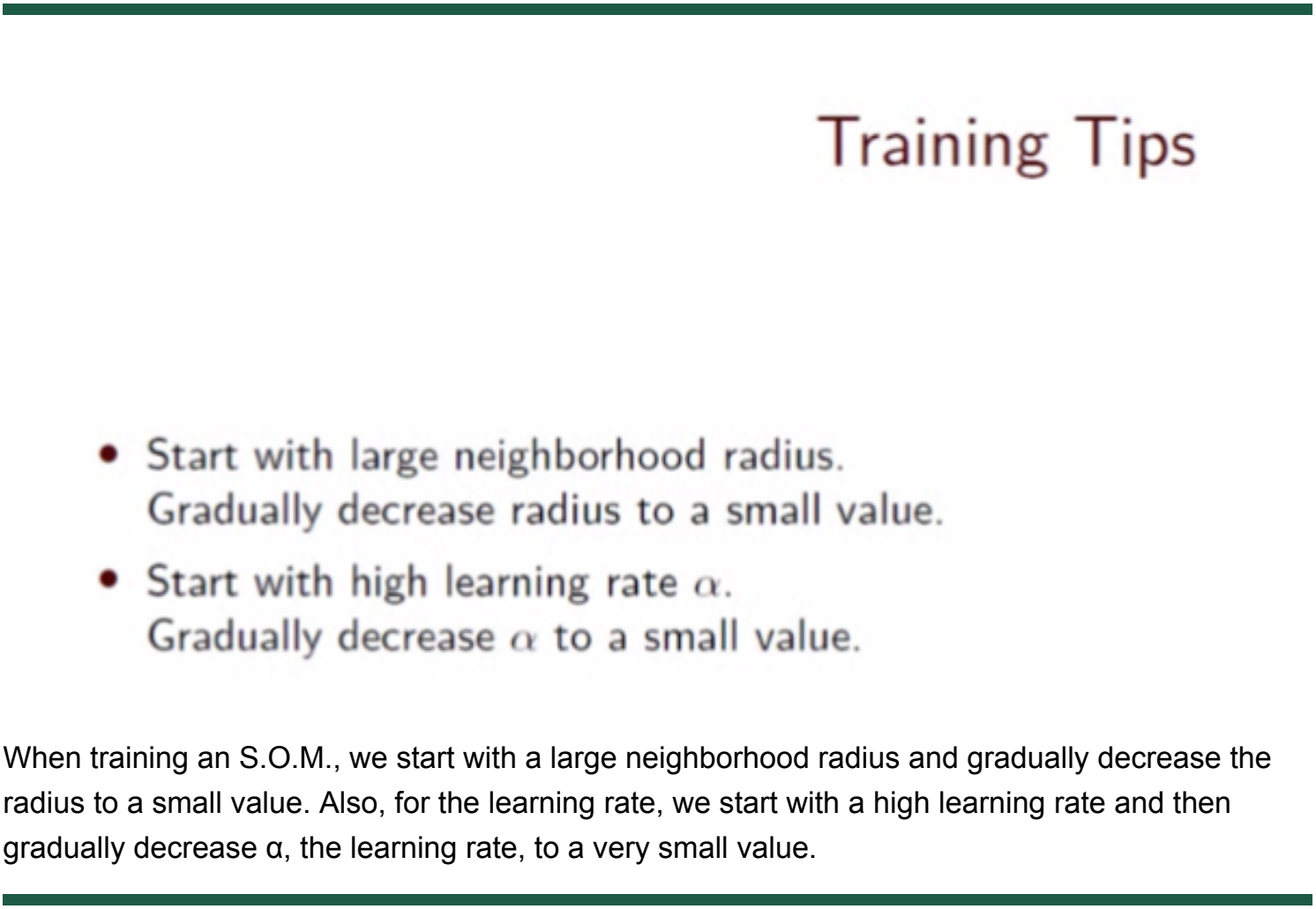
We can also make it simple and make it into a cylinder shape.

Typical Neighborhood Functions



- Gaussian: $\Lambda(i(\mathbf{x}), j) = exp(-|\mathbf{r}_j - \mathbf{r}_{i(\mathbf{x})}|^2/2\sigma^2)$
  where $\mathbf{r}_j$ is the coordinate of unit j on the grid.
- Flat: $\Lambda(i(x), j) = 1$ if $|\mathbf{r}_j - \mathbf{r}_{i(\mathbf{x})}| \leq \sigma$, and 0 otherwise.
- σ is called the **neighborhood radius**.

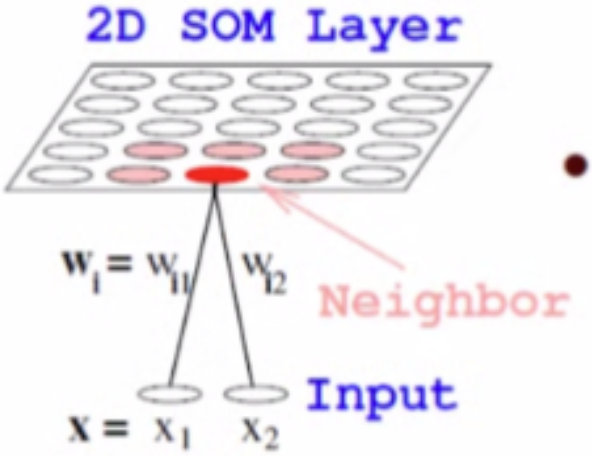
Here is an example. We have a cylinder center that is the location of the best matching unit. And if we are within the radius, the neighborhood function value is 1; otherwise, it is 0.



When training an S.O.M., we start with a large neighborhood radius and gradually decrease the radius to a small value. Also, for the learning rate, we start with a high learning rate and then gradually decrease α, the learning rate, to a very small value.

Properties of SOM

- **Approximation of input space.**
  Maps continuous input space to discrete output space.
- **Topology preservation.**
  Nearby units represent nearby points in input space.
- **Density mapping.**
  More units represent input space that are more frequently sampled.



Here are the three main properties of S.O.M. It will approximate input space, and it will preserve the topology. It will also do density mapping.

The long S.O.M. connection weights will map the continuous input space into these discrete output spaces.

Because nearby units are reinforced to change their weight together toward the input that is mapped to the best matching unit, the topology of the input space will be preserved. So, that means nearby units represent nearby points in the input space.

Lastly, it will conduct density mapping. Thus, if we have a region of the input space that has more samples in it than other parts of the input space, then more units will be allocated to represent that more dense subspace.

Performance Measures

- **Quantization Error**
  Average distance between each data vector and its BMU.

$$\epsilon_Q = \frac{1}{N}\sum_{j=1}^{N} \| \mathbf{x}_j - \mathbf{w}_{i(\mathbf{x}_j)} \|$$

- **Topographic Error**
  The proportion of all data vectors for which first and second BMUs are not adjacent units.

$$\epsilon_T = \frac{1}{N}\sum_{j=1}^{N} u(\mathbf{x}_j),$$

$u(\mathbf{x}) = 1$ if the 1st and 2nd BMUs are not adjacent
$u(\mathbf{x}) = 0$ otherwise.

Although there is no performance measure such as accuracy for S.O.M. because it is an unsupervised learning algorithm, there are certain performance measures to measure the quality of the learned mapping. The first measure is the quantization error. The second one is the topographic error.

Performance Measures

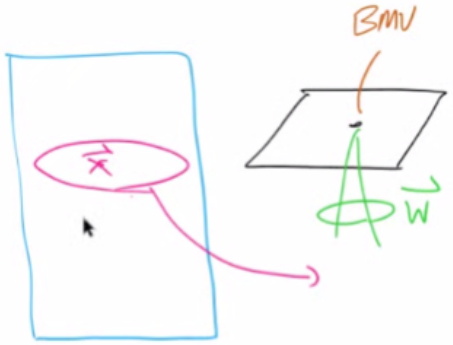
- **Quantization Error**
  Average distance between each data vector and its BMU.

$$\epsilon_Q = \frac{1}{N}\sum_{j=1}^{N} \| \mathbf{x}_j - \mathbf{w}_{i(\mathbf{x}_j)} \|$$

- **Topographic Error**
  The proportion of all data vectors for which first and second BMUs are not adjacent units.

$$\epsilon_T = \frac{1}{N}\sum_{j=1}^{N} u(\mathbf{x}_j),$$

$u(\mathbf{x}) = 1$ if the 1st and 2nd BMUs are not adjacent
$u(\mathbf{x}) = 0$ otherwise.



First, let us look at the quantization error. Here what we do is we go through all the inputs, and for each of the input samples, we find the best matching unit, compare the input vector and the best matching units weight vector, and find the Euclidean distance between these two. Thus, if these are the same, then this will be zero.

We loop through all samples. So, for this input, it will have its own best-matching unit. For the next input will have another best-matching unit. We find the distance between this input and their best matching unit, and we sum it up and divide it by *N*, where *N* is the number of samples in our dataset. And this is the quantization error.

So, this is a measure of how well our S.O.M. map represents the dataset.

Performance Measures

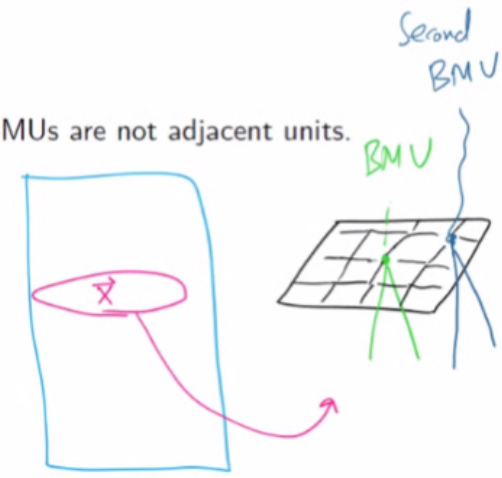
- **Quantization Error**
  Average distance between each data vector and its BMU.

$$\epsilon_Q = \frac{1}{N} \sum_{j=1}^{N} \| \mathbf{x}_j - \mathbf{w}_{i(\mathbf{x}_j)} \|$$

- **Topographic Error**
  The proportion of all data vectors for which first and second BMUs are not adjacent units.

$$\epsilon_T = \frac{1}{N} \sum_{j=1}^{N} u(\mathbf{x}_j),$$

$u(\mathbf{x}) = 1$ if the 1st and 2nd BMUs are not adjacent
$u(\mathbf{x}) = 0$ otherwise.

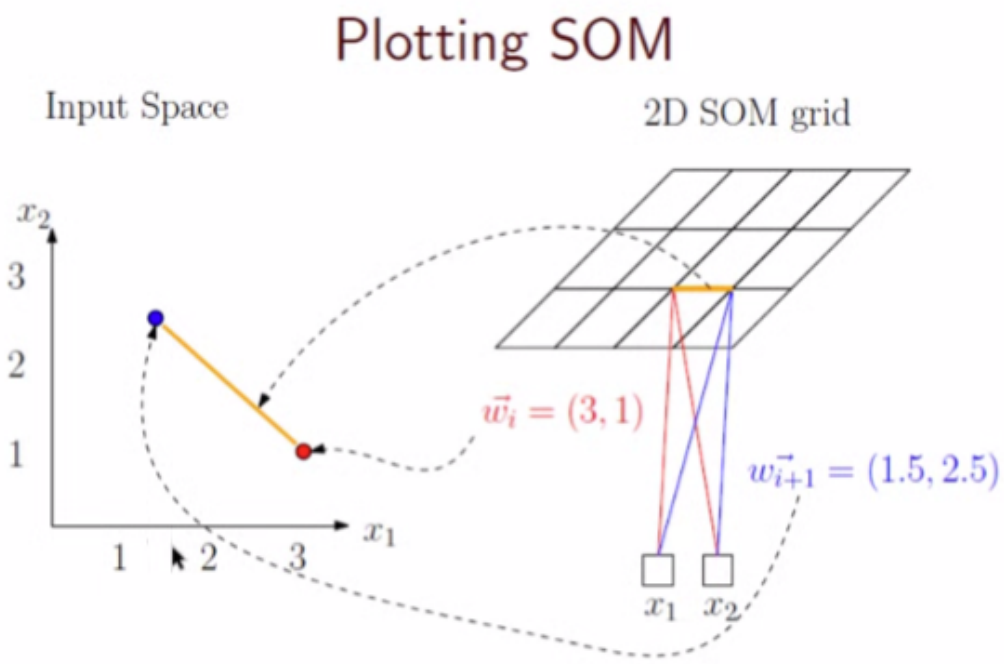


Topographic error measures how similar a single unit weight vector is compared to its direct neighbors.

So again, we go through the entire data set and take a sampling of the input vector. So, here is *x*, and we find the best matching unit and the second best matching unit among all of these weight vectors. If these happen to be connected on the grid directly being exact neighbors, then this *u* value is zero.

Otherwise, if the first and second best matching units are not adjacent on the grid, this value would be 1.

Again, we divide the sum with 1 over *N* to find the average mismatch in this adjacency requirement, which is the topographic error. In summary, it measures how neighboring units represent similar input.



- Step 1: Plot weight vectors in the input space.
- Step 2: Connect them according to their location in the SOM grid.

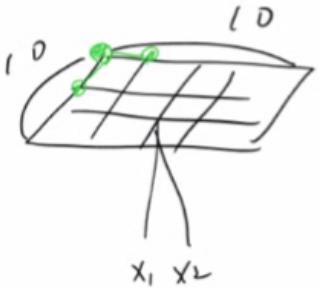
As we have seen earlier, we can plot the connection weights or the weight vectors in the same grid into the input space. For example, this unit here has a weight vector of (3, 1), and we can plot it in the input space (3, 1). The exact neighbor of that unit has a weight vector of (1.5, 2.5), so we can plot it in the input space like this.

Finally, we can connect them with a straight line because they are exact neighbors.

Using this kind of technique, we can plot the entire configuration of the 2-D grid in the input space.

Example: 2D Input / 2D Output

- Train with uniformly random 2D inputs.
  Each input is a point in Cartesian plane.
- Nodes: reference vectors (x and y coordinate).
- Edges: connect immediate neighbors on the map.



Here is an example of a 10 × 10 grid with two-dimensional input. When we initialize all of these vectors to be random and connect them according to the same plotting technique that we saw in the previous slide, we will end up with something like that.

Let us consider this. This might be one of these corners. Let us say it corresponds to this particular corner in the grid. Then we can identify these two exact neighbors by following these two edges. So, one ends up here; the other ends up here, and this now would be the weight vector location for that unit.

The x_1, x_2 value might be the weight vector location for this unit.

Example: 2D Input / 2D Output

- Train with uniformly random 2D inputs.
  Each input is a point in Cartesian plane.
- Nodes: reference vectors (x and y coordinate).
- Edges: connect immediate neighbors on the map.

Initially, the neighborhood radius is very large, and the learning rate is big. So, all of these would move toward the input.

For example, if this input were sampled, all that would move toward that particular input location.

It looks something like that. Then if another input is sampled over there, this whole mess will move toward that location.

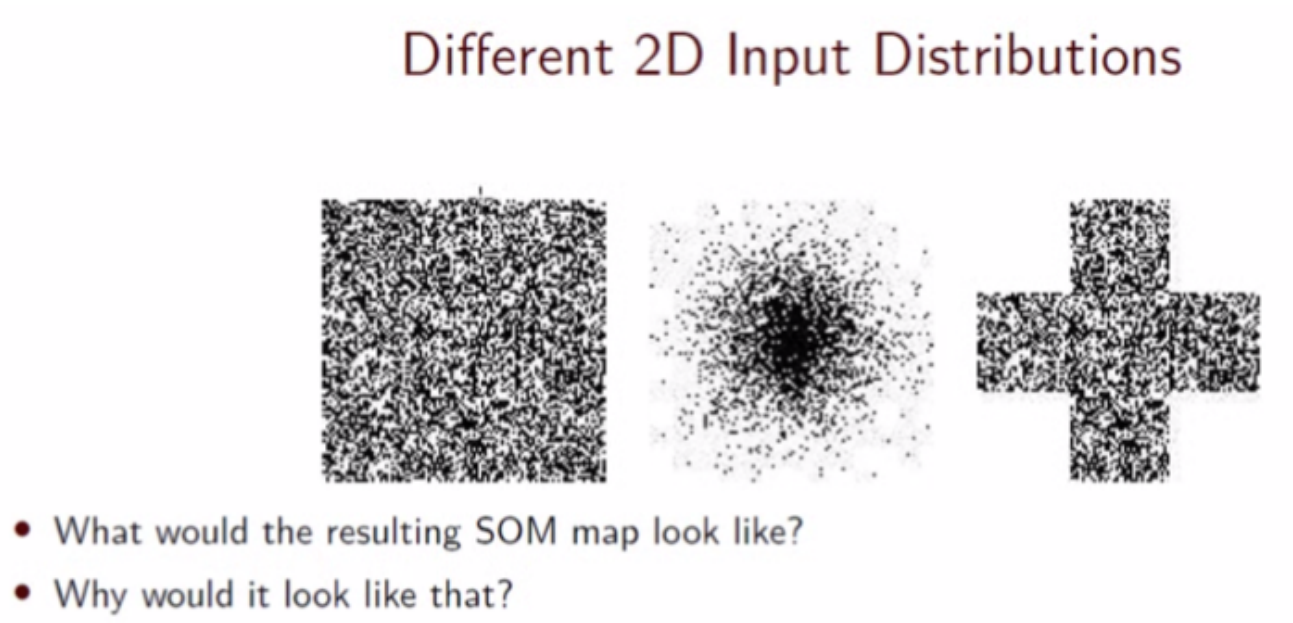
Example: 2D Input / 2D Output

- Train with uniformly random 2D inputs.
  Each input is a point in Cartesian plane.
- Nodes: reference vectors (x and y coordinate).
- Edges: connect immediate neighbors on the map.

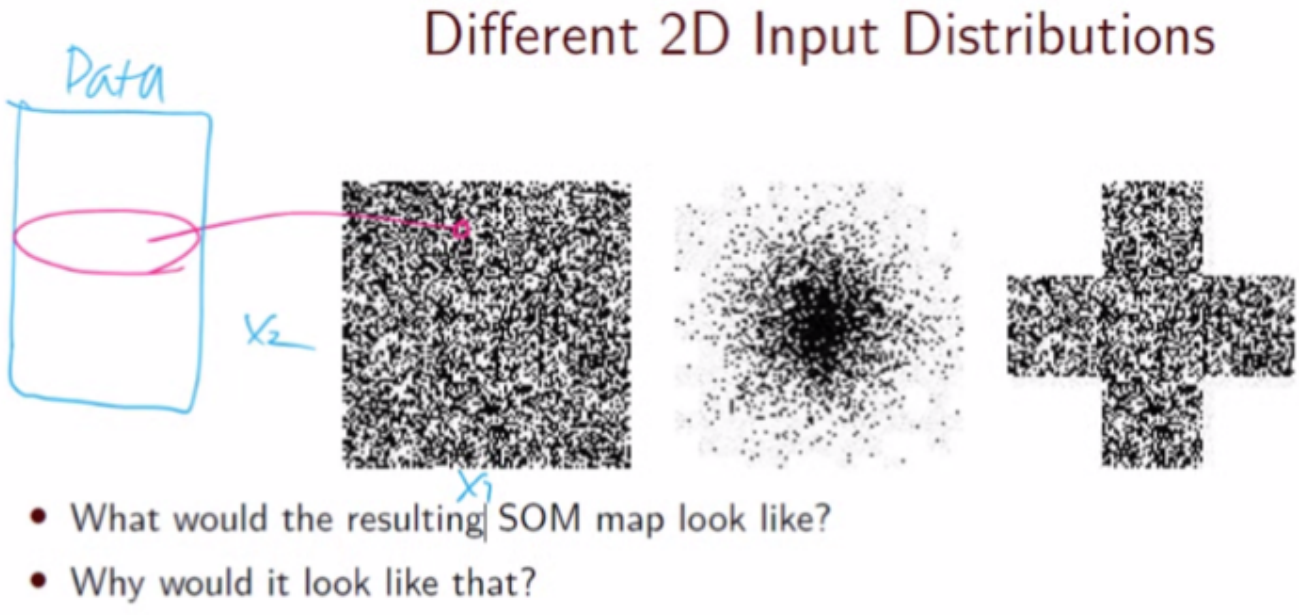
As the neighborhood radius reduces and the learning rate reduces gradually, what will happen is that these units will now form this kind of grid, and as the neighborhood radius is further reduced,

then they will span out to fill the input space.

This particular example was when the dataset consisted of random x_1, x_2 values that filled the entire input range.

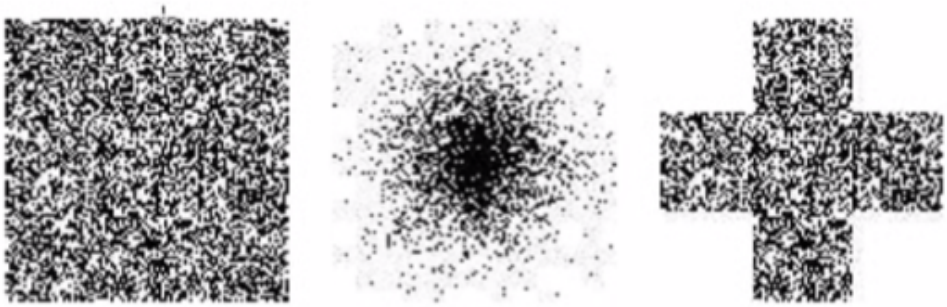


Here is an example that may have been used for the previous result.



Each dot in this plot might correspond to one row in the data set. x_1 and x_2 values are shown here.

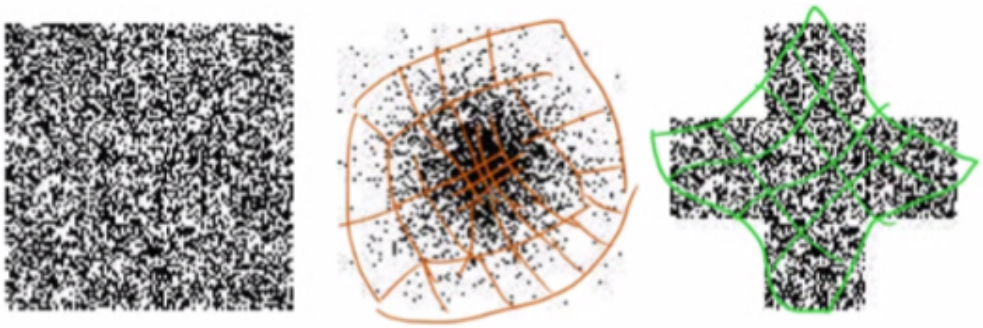
Different 2D Input Distributions



- What would the resulting SOM map look like?
- Why would it look like that?

When the input has a different structure like this with more dense samples in the middle or in the case of this kind of cross-shaped area where samples come from, then the learned grid might look very different.

Different 2D Input Distributions



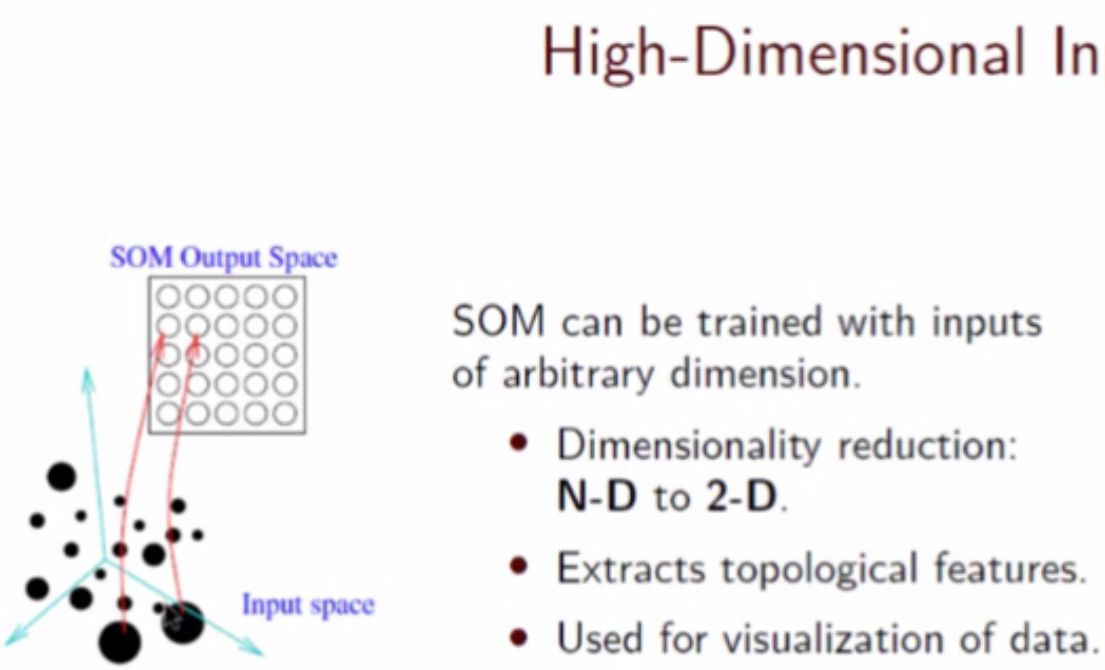
- What would the resulting SOM map look like?
- Why would it look like that?

For example, for this data set in the middle, more samples are concentrated in the middle so that more units will be allocated to the region.

Thus, the distance between the grid locations would be shorter. Whereas if we go outside, then

the distance between these units would be longer or larger than those in the middle. Also, if we count the units within this region, there are many more compared to this region of equal area.

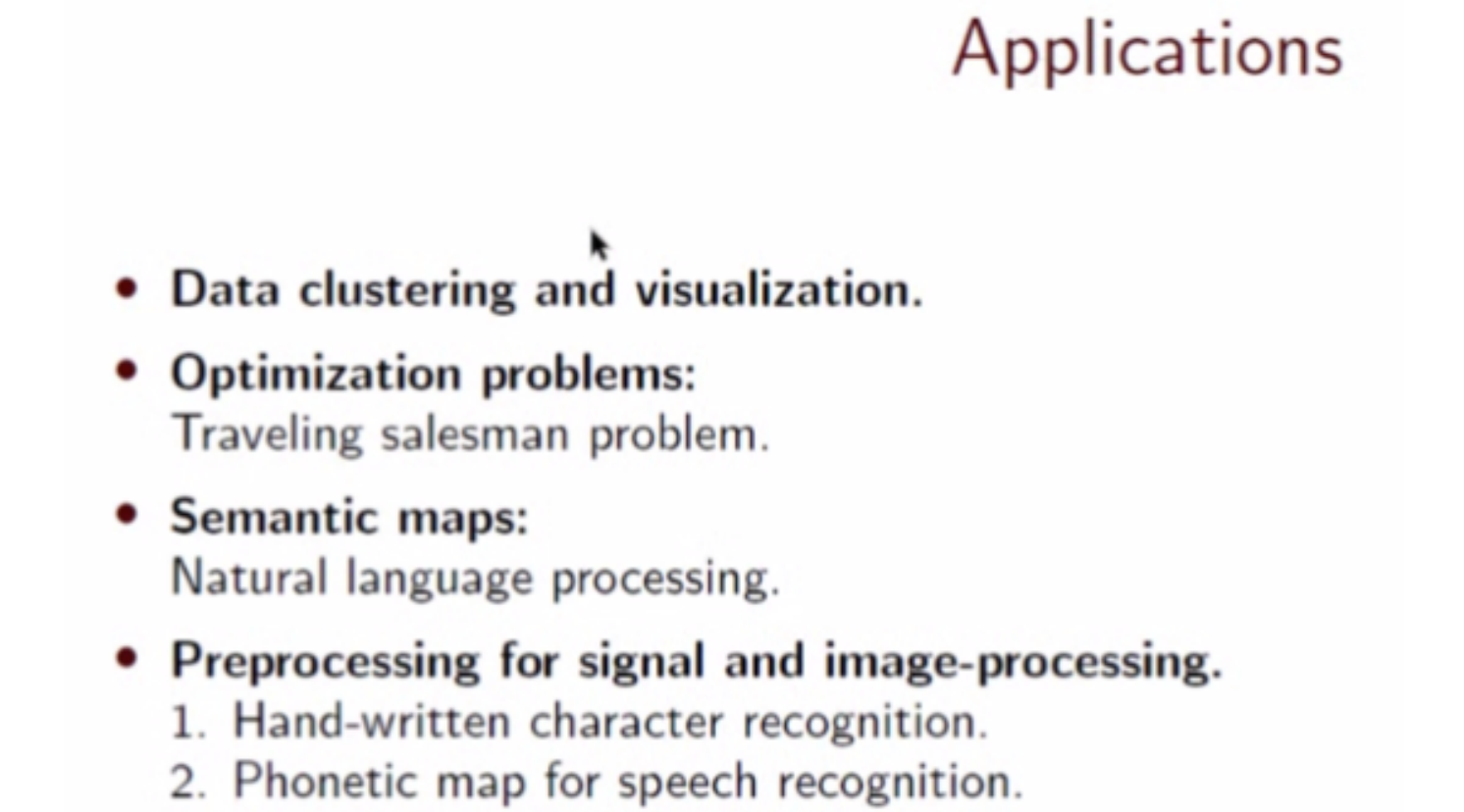
Next, for input with this kind of cross shape, considering that no input samples are coming from these blank regions, none of these units would be allocated, and we will end up with a grid that looks like that.



S.O.M. can get really interesting when we use high-dimensional inputs, not just 1-D or 2-D, or even 3-D. As we have seen at the very beginning of the machine learning modules, we saw that a typical image, for example, a 50 × 50 image, is a very high-dimensional vector. It may be a 2,500-dimensional vector, in fact. And we can train S.O.M. using a bunch of images like that.

Then what we can see is how these different images living in this 2,500-dimensional space occupy and map this 2-D space.

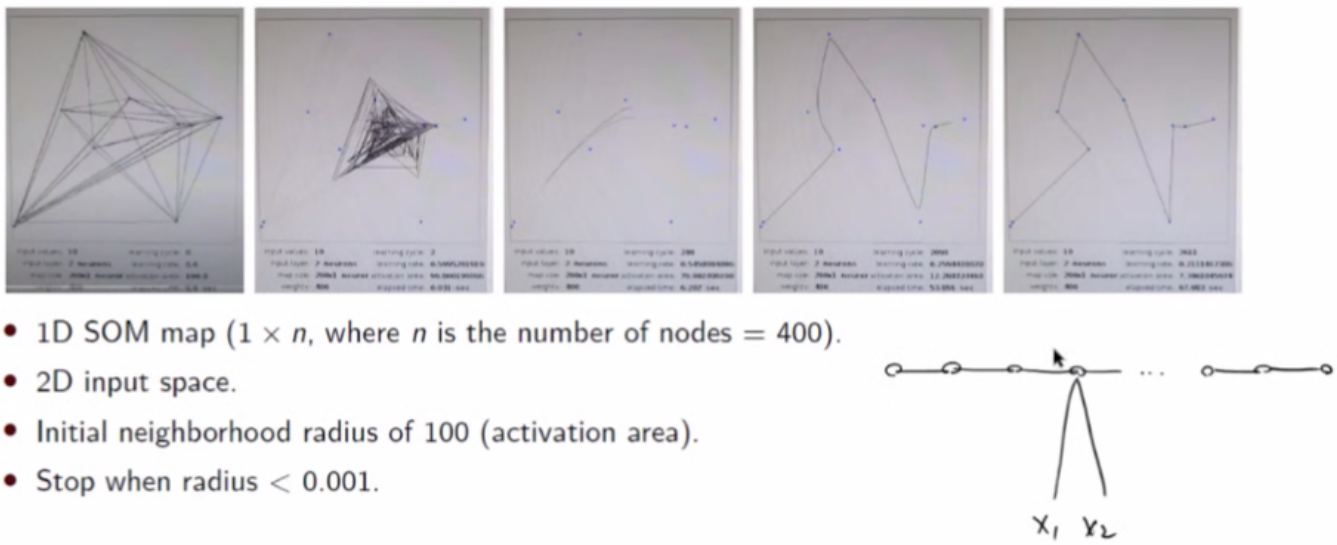
Using this, we can effectively reduce the dimensionality of these 2,500 dimensions down to 2-D. Using this, we can extract the topological properties in the input dataset. We can also use this for visualization.



S.O.M. has been used in many applications such as data clustering and visualization, optimization problems such as the traveling salesman problem, semantic mapping, or feature mapping. This has been used for natural language processing. Also, as a pre-processing step for signal and image processing. For example, handwritten character recognition or phonetic mapping for speech recognition.

I created some demos based on Jochen Froehlich's neural networks with Java. We can find out the source code **here** **(http://rfhs8012.fh-regensburg.de/~saj39122/jfroehl/diplom/e-index.html)**

(expired link).



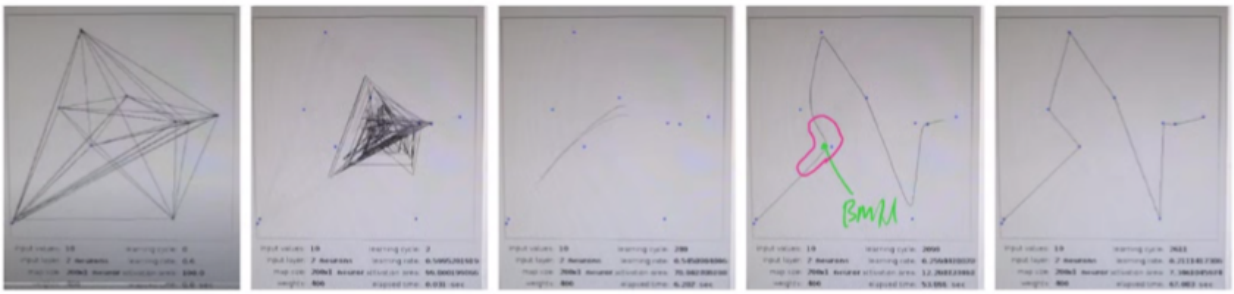
Let us look at this simple demo. This is a demo of the traveling salesman problem. Here we have a 400-unit 1-D grid. So, it is just a 1-D chain of 400 units. And each of these has a 2-dimensional weight vector.

Suppose in the dataset, we had 10 units or 10 samples.

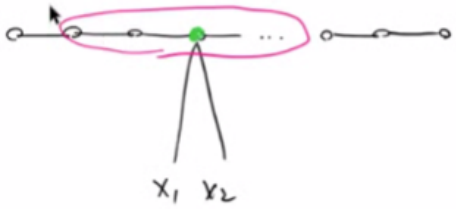
We initialize the weight vectors randomly so that they attach to one of the 10 samples. Then we will get something like that. Then we start training. Again, initially, the neighborhood radius is very large. So, when an input appears here, then all of these would bunch toward it. But eventually, when the learning rate and the neighborhood radius are decreased, then they will form a nice small trace like that.

Thus, at this point, when input is sampled, then all of these will move toward it. When another input is sampled, then this whole trace will move toward it. But near the end of the training session, what happens is when input is sampled, only those on the chain that have the rate vector close to the best matching unit's location will be updated.

SOM Demo: Traveling Salesman Problem



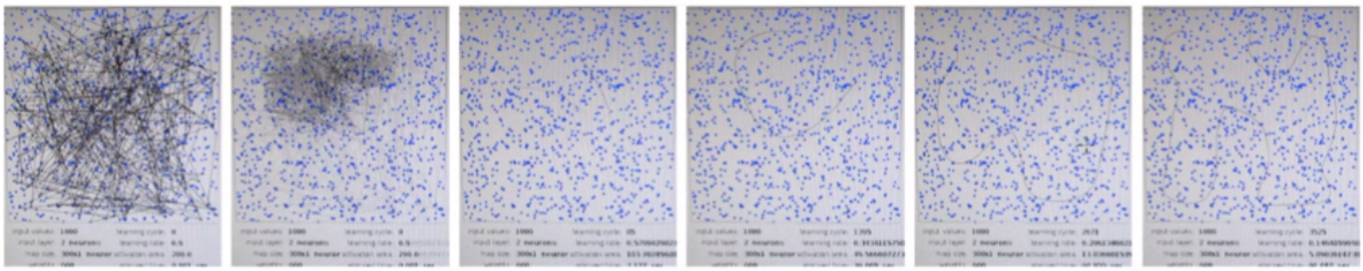
- 1D SOM map ($1 \times n$, where n is the number of nodes $= 400$).
- 2D input space.
- Initial neighborhood radius of 100 (activation area).
- Stop when radius < 0.001.



If this particular one here were the best matching unit, then its immediate neighbor, only these, would move toward that input that was just sampled.

In the end, we can see that these 400 units form this nice trajectory across these 10 input samples.

SOM Demo: Space Filling in 2D



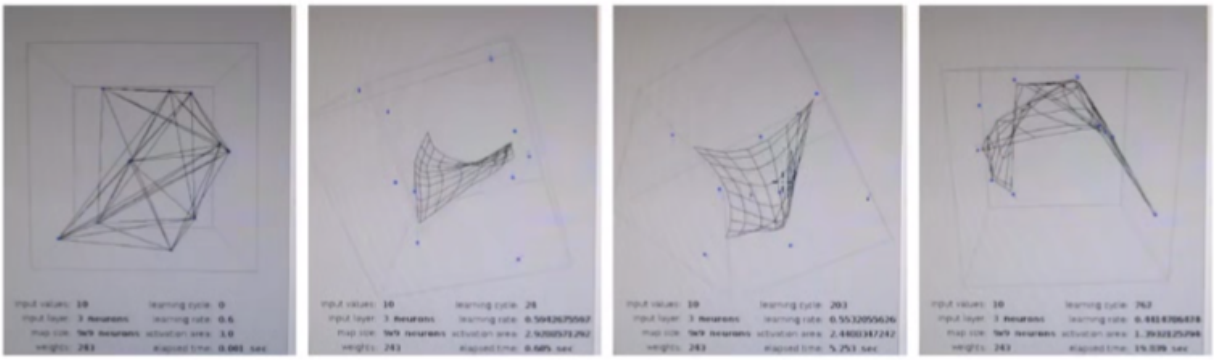
- 1D SOM map ($1 \times n$, where n is the number of nodes $= 600$).
- 2D input space. 1000 input points.
- Initial neighborhood radius of 200.
- Stop when radius < 0.001.

Here is an example of a 1-D chain with 600 nodes in it. Again, when we randomly initialize these weights, we get something like a bird's nest, and in this case, we had a thousand input points randomly scattered around in the input space. Again, initially, when the neighborhood radius is

large, and the learning rate is large, all of these would tend toward the current input that was sampled.

Gradually, they will form this small trace, and these will move around, depending on which part of the input space was sampled, but gradually, only a certain part, a small subset of nodes in the 1-D chain, will move toward the input that is sampled, and as the neighborhood radius is further reduced, we sample in this case only this region will move toward the input, so there will be these protrusions into the input space, which is not very well represented. Eventually, we will see something like this. This is commonly known as the space-filling curve.

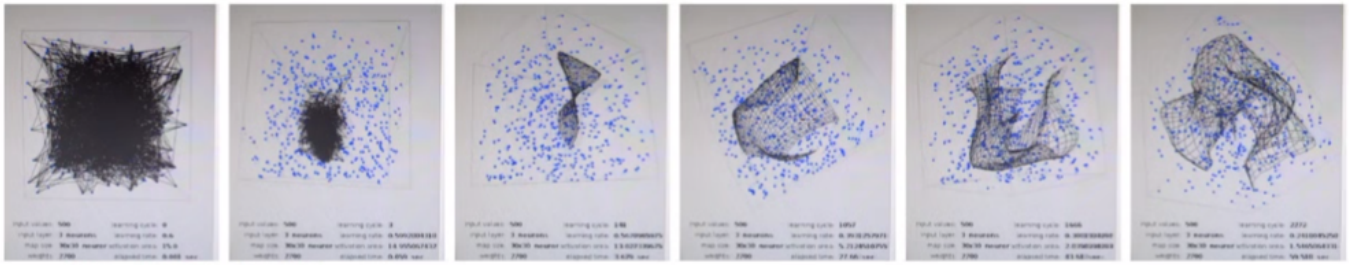
SOM Demo: Representing 3D with 2D



- 2D SOM map ($n \times n$, where n is the number of nodes, 9×9).
- 3D input space. 10 input points.
- Initial neighborhood radius of 3.
- Stop when radius < 0.001.

Next is a 2-D grid with 3-D input space. So, the input does not need to be 2-D. It can be 3-D or 4-D, or even higher, like 2,500 dimensions. In this case, let us see what happens when we want to represent 3-D space as a 2-D grid. And here we had 10 input points. As we can see from this, there are 10 blue dots in the 3-D code system. Initially, it is random, and then gradually, the 2-D grid will occupy the 3-D space so that it will cover the points that exist in the input dataset.

SOM Demo: Space Filling in 3D



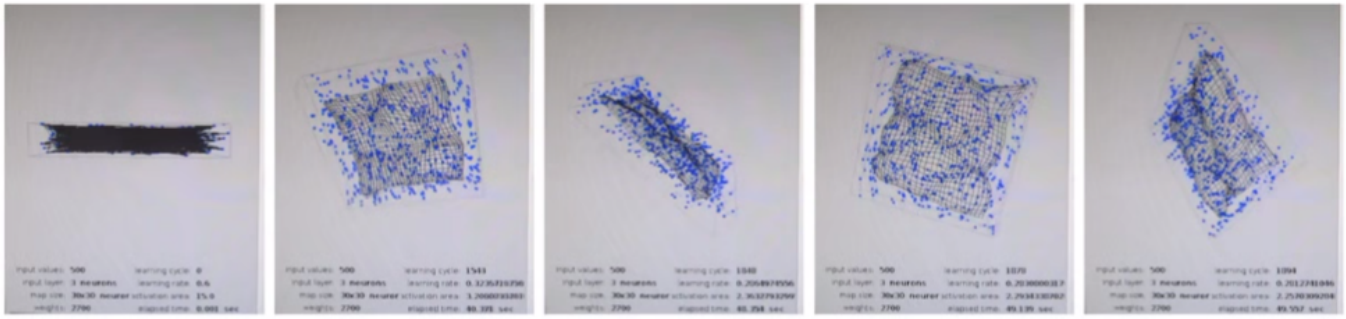
Using Fröhlich's SOM applet:

- 2D SOM map ($n \times n$, where n is the number of nodes, 30×30).
- 3D input space. 500 input points.
- Initial neighborhood radius of 15.
- Stop when radius < 0.001.

This is an example of space-filling in 3-D. We start again with a 2-D map or 2-D grid, and we have a dense sampling of data points within this 3-D cube. Again, initially, all of these tend towards the input that is currently presented, but gradually they will fan out to fill the space.

Eventually, we will get this kind of crumpled piece of paper type of representation.

SOM Demo: Space Filling in 3D (Pizza Box)



- 2D SOM map ($n \times n$, where n is the number of nodes, 30×30).
- 3D input space. 500 input points.
- Initial neighborhood radius of 15.
- Stop when radius < 0.001.

Finally, here is a variation of space-filling in 3-D. Here we have a pizza box full of data points in it, but as we can see, one of these axes is very short compared to the other two axes. Initially, the 2-D grid will form a 2-D sheet, but as the training goes on, it will undulate to represent the thickness of the pizza box.

[Video is shown]

Let us look at some demos. This is the traveling salesman case.

We can see that this wiggly line is jumping around, and all of them are moving. As we can see, one part of it may be protruding, and that is because the particular input was sampled.

We can see the activation area corresponding to the neighborhood radius is gradually decreasing. In this case, we had a 200-node chain.

Now the neighborhood radius is small enough so that when an input is sampled, only a small part of that entire chain is moved toward that input.

[Video is shown]

Now let us look at the 2-D grid with 2-D input space.

Again, the whole grid is moving around as the inputs are sampled.

This is a 30 × 30 grid, and now the neighborhood radius is down to seven. As we can see, because the neighborhood radius is smaller than 30, what is happening is only a small part of that grid is moving toward the input.

Let me pause this for a moment. What is interesting here is that in this region, there are more samples per unit area compared to that region. As we can see, more units are being moved toward that region compared to this neighboring region, where there are not too many input samples coming from.

The same for here is pretty dense, and here it is pretty sparse, and so on. Let me continue.

That is the end of it.

[Video is shown]

Next, let us look at the 1-D grid and 2-D input. Here we have 300 nodes in the 1-D grid or the chain.

Again, we can see the wiggly line jumping across the entire input space.

Again, where they jump is dependent on which input was sampled.

This is a 300-unit case, and because the neighborhood radius is down to about 50, only a small fraction of that whole chain will move toward the sampled input.

As the neighborhood radius decreases further, what will happen is that these protrusions will

appear to fill this input space.

As we can see now, this middle region of the input space is represented, and it will further contort and protrude toward the region. Again, that is not being well represented.

[Video is shown]

Now let us look at space-filling for the 3-D space with a 2-D map.

Again, the samples come from this cube uniformly and randomly generated.

Then we have a 30 × 30 2-D grid trying to map this input space.

Again, we had these large regions, and they are collapsing toward the middle as the radius is being reduced.

As we can see near the end, we see something like a crumpled piece of paper filling the 3-D space.

[Video is shown]

Finally, let us look at this space-filling 3-D for this pizza box shape.

Initially, it will just form this 2-D sheet, but gradually will also try to model or represent the thin height of this pizza box.

When it turns to the side, we can see the undulation.

SOM Application: Semantic Map

TABLE 9.3 Animal Names and Their Attributes

Animal		Dove	Hen	Duck	Goose	Owl	Hawk	Eagle	Fox	Dog	Wolf	Cat	Tiger	Lion	Horse	Zebra	Cow
is	small	1	1	1	1	1	1	0	0	0	0	1	0	0	0	0	0
	medium	0	0	0	0	0	0	1	1	1	1	0	0	0	0	0	0
	big	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1	1
has	2 legs	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0
	4 legs	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1	1
	hair	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1	1
	hooves	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1
	mane	0	0	0	0	0	0	0	0	0	1	0	0	1	1	1	0
	feathers	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0
likes to	hunt	0	0	0	0	1	1	1	1	0	1	1	1	1	0	0	0
	run	0	0	0	0	0	0	0	0	1	1	0	1	1	1	1	0
	fly	1	0	0	1	1	1	1	0	0	0	0	0	0	0	0	0
	swim	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0

- SOM can be trained with high-dimensional vectors representing multi-attributed entities such as animals.
- The resulting map could show interesting relationship among the animals.

Far in the demonstrations, we have only seen the low-dimensional input case, either 2-D or 3-D

input. However, the utility of S.O.M. really shines when we have high-dimensional input space.

For example, this data set has 13 dimensions. For each input data point, for example, dove, hen, and duck, we have 13 binary values to describe the feature of this animal, is it small, medium, or big? Does it have two legs or four legs? Does it have hair, hooves, mane, or feathers? Does it hunt? Does it run, fly, or swim?

These vectors represent each of these different animals, and we can train an S.O.M. map using this data set.

The resulting map will tell us interesting relationships among these animals. And let us look at the trained map in the next slide.

Semantic Map: BMUs

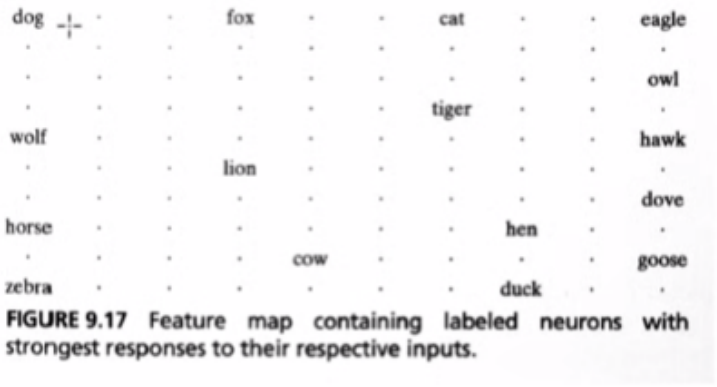


FIGURE 9.17 Feature map containing labeled neurons with strongest responses to their respective inputs.

- The sixteen inputs are mapped to their best matching units on the map.
- Try guessing what would their neighboring units' weight vector would look like.

Here is the trained map. We trained the 10 × 10 map.

Semantic Map: BMUs

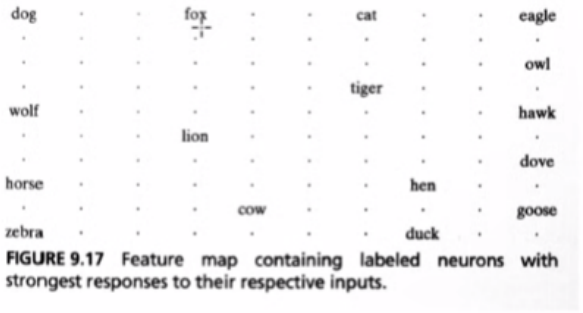
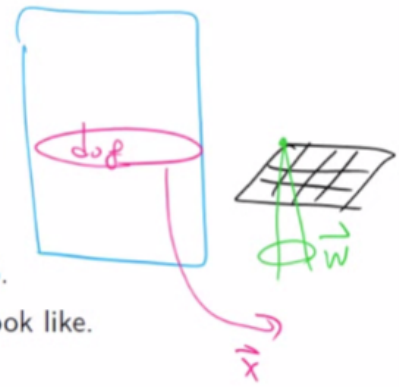


FIGURE 9.17 Feature map containing labeled neurons with strongest responses to their respective inputs.



- The sixteen inputs are mapped to their best matching units on the map.
- Try guessing what would their neighboring units' weight vector would look like.

For each input in the dataset, we find the best matching unit by comparing the weight vector with that input vector. We plot that particular label at the location of the best matching unit. For example, for the dog input, the top left corner of the grid was the best matching unit, so we wrote down "dog" there.

Next, we pick "fox," and this unit somewhere around the middle is the best matching unit. We write it down, and we do the same for all of these 16 different animals.

Semantic Map: BMUs

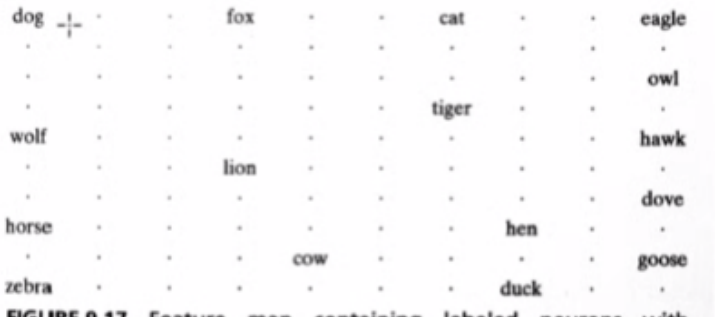


FIGURE 9.17 Feature map containing labeled neurons with strongest responses to their respective inputs.

- The sixteen inputs are mapped to their best matching units on the map.
- Try guessing what would their neighboring units' weight vector would look like.

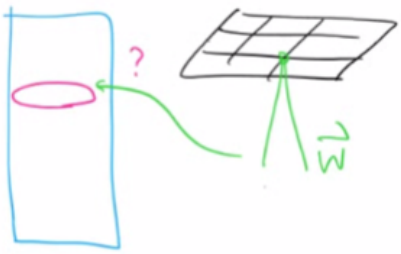
What is interesting is that these animals are related. for example, "dog," "fox," and "wolf" are all in the canine family. "Cat," "tiger," and "lion" are in the feline family. "Eagle," "owl," and "hawk" are birds of prey. "Dove," "goose," "hen," and "duck" are peaceful birds. "Cow" and "horse" are cattle, and "zebra" is the wild version of the horse.

Also, foxes, lions, wolves, and tigers are predators. They are closely related or close by on the map, and so on. There are lots of interesting relationships that are represented on this map.

Semantic Map: Probed Map

dog	dog	fox	fox	fox	cat	cat	cat	eagle	eagle
dog	dog	fox	fox	fox	cat	cat	cat	eagle	eagle
wolf	wolf	wolf	fox	cat	tiger	tiger	tiger	owl	owl
wolf	wolf	lion	lion	lion	tiger	tiger	tiger	hawk	hawk
wolf	wolf	lion	lion	lion	tiger	tiger	tiger	hawk	hawk
wolf	wolf	lion	lion	lion	owl	dove	hawk	dove	dove
horse	horse	lion	lion	lion	dove	hen	hen	dove	dove
horse	horse	zebra	cow	cow	cow	hen	hen	dove	dove
zebra	zebra	zebra	cow	cow	cow	hen	hen	duck	goose
zebra	zebra	zebra	cow	cow	cow	duck	duck	duck	goose

FIGURE 9.18 Semantic map obtained through the use of simulated electrode penetration mapping. The map is divided into three regions representing: birds, peaceful species, and hunters.



- Find out, for each SOM unit, which animal it responds maximally.
- For this kind of probe, every unit will have an animal assigned, even when they are not the BMU for any of these animals.
- Note the partitioning of the map into coherent categories.

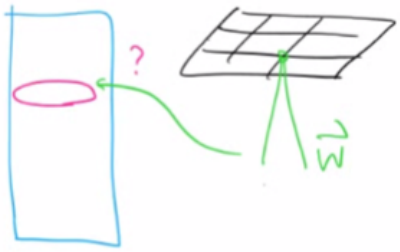
One additional way of looking at what the map is doing is called probing. For each weight vector in the grid, what we can do is find which is the best matching input. In the previous slide, we went from the input and found the best matching unit. But here we ask the reverse question. For example, in this unit with no label in the previous slide, we look at the weight vector and find which of the inputs best matches this particular weight vector.

So, this weight vector may not be the best matching unit for any of these inputs, but we can find which one is the closest. Using this, we can label all of these units on the grid, and something very interesting appears. Thus, these four units, exact neighbors, all represent "dog," and these all represent "fox."

Semantic Map: Probed Map

dog	dog	fox	fox	fox	cat	cat	cat	eagle	eagle
dog	dog	fox	fox	fox	cat	cat	cat	eagle	eagle
wolf	wolf	wolf	fox	cat	tiger	tiger	tiger	owl	owl
wolf	wolf	lion	lion	lion	tiger	tiger	tiger	hawk	hawk
wolf	wolf	lion	lion	lion	tiger	tiger	tiger	hawk	hawk
wolf	wolf	lion	lion	lion	owl	dove	hawk	dove	dove
horse	horse	lion	lion	lion	dove	hen	hen	dove	dove
horse	horse	zebra	cow	cow	cow	hen	hen	dove	dove
zebra	zebra	zebra	cow	cow	cow	hen	hen	duck	goose
zebra	zebra	zebra	cow	cow	cow	duck	duck	duck	goose

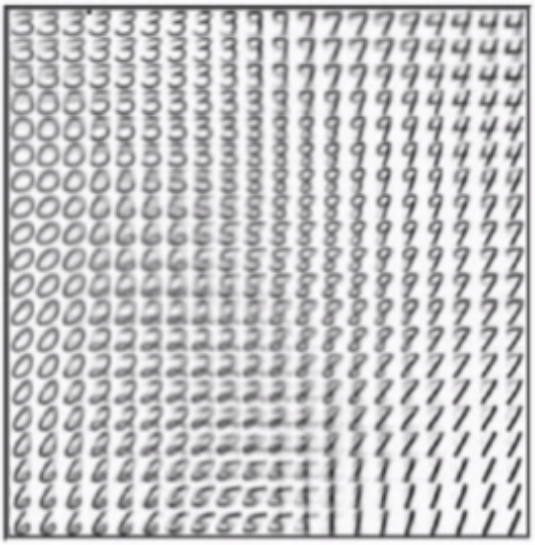
FIGURE 9.18 Semantic map obtained through the use of simulated electrode penetration mapping. The map is divided into three regions representing: birds, peaceful species, and hunters.



- Find out, for each SOM unit, which animal it responds maximally.
- For this kind of probe, every unit will have an animal assigned, even when they are not the BMU for any of these animals.
- Note the partitioning of the map into coherent categories.

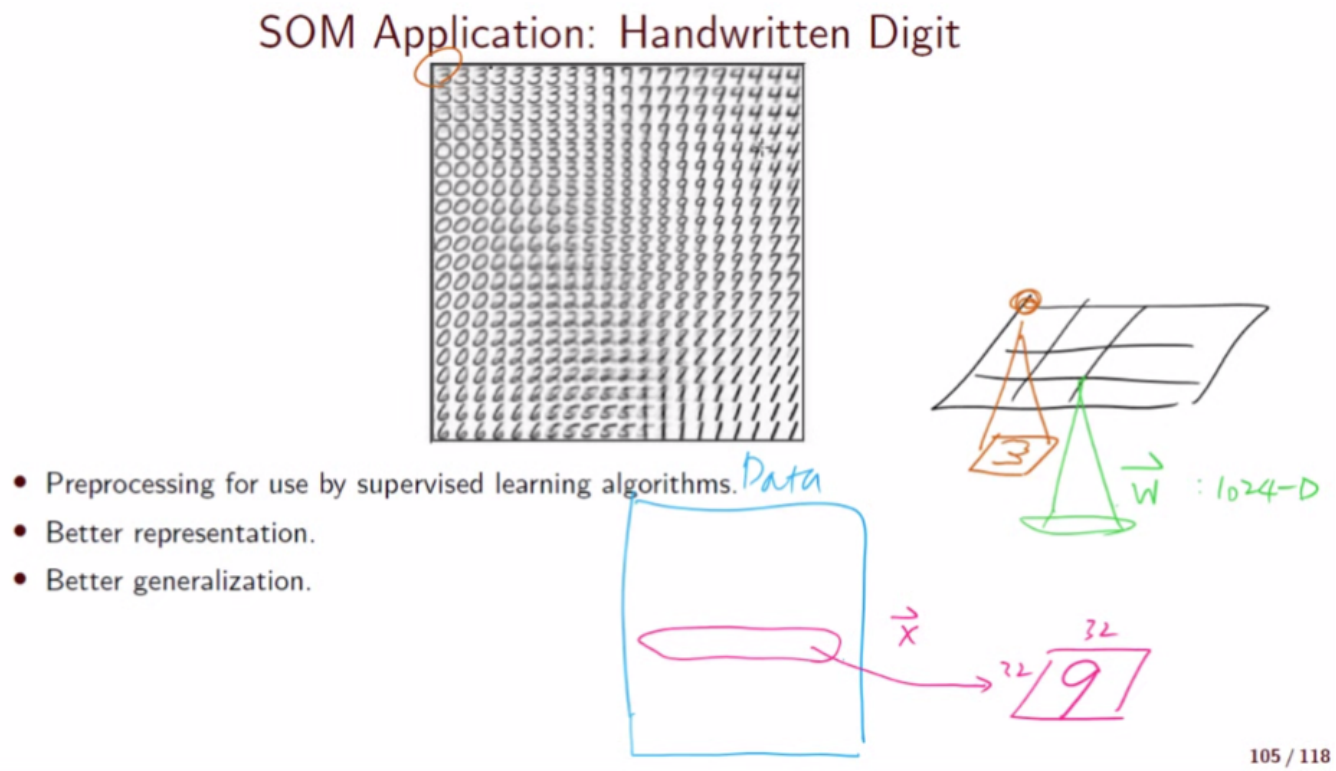
We can draw explicit contours around these animals. And we can see this is the dog region. Here is the wolf region. Here is the fox region. Cat, tiger, and lion region. And we can draw boundaries between these animal regions. So, this boundary will divide between these four-legged animals and birds. This boundary will divide these animals into predators and more peaceful herbivore animals. Again, draw a boundary between these herbivore animals and birds.

SOM Application: Handwritten Digit



- Preprocessing for use by supervised learning algorithms.
- Better representation.
- Better generalization.

Finally, here is an example of applying S.O.M. to handwritten digit data.



In this case, we had a dataset of handwritten digits. Each digit was a 32 × 32 image. So, if we make it into a single row, that will be a 1,024-dimensional vector.

Hence each weight vector for the unit in the S.O.M. grid would be 1024 dimensions. The plot here shows the weight vector represented as a 32 × 32 image. Thus, this particular location would correspond to the weight vector for that unit.

As shown here, this is the corner unit, and when we plot the weight vector, the 1024-dimensional weight vector as a 32 × 32 image, then we see something like a digit 3.

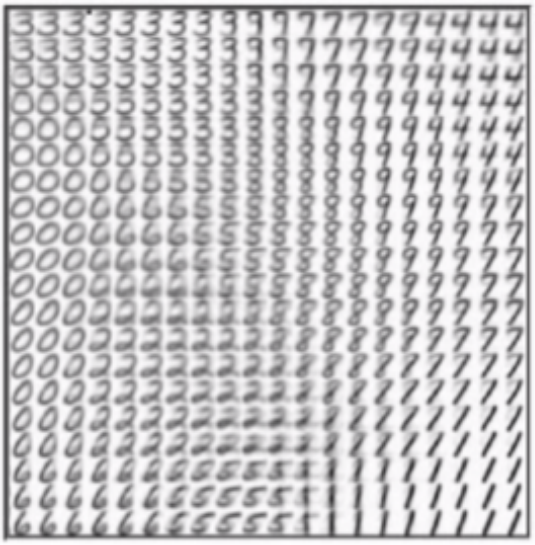
The interesting thing is that these neighboring units around that corner unit all represent something that looks like a three. And then this 3 gradually changes over to 9, and then 7, and then 4. So, this region at that corner corresponds to the number 4, and then nearby that is the region for 7, the region for 9, and so on.

Again, we have another redundant region for 7 with different angles. Then this will transition gradually into the region at the bottom right, representing the digit 1.

With this kind of map, we can visualize the similarity and the relationship between the different digits in terms of their shape. So, 1 is similar to 7 and 9, 0 is similar to 6, and a fat 0 might be similar to a fat 3, and so on. An 8 might be similar to a 9. Nine might be similar to 4, and so on.

Once we have this kind of representation, we can get the activation value based on the Euclidean distance given a particular input, and that itself can serve as a good representation, and we can feed this into a subsequent supervised learning algorithm to get better performance and better generalization.

SOM Application: Handwritten Digit



- Preprocessing for use by supervised learning algorithms.
- Better representation.
- Better generalization.

This is the end of this lecture on S.O.M. In the next lecture, we will look at recurrent neural networks and genetic algorithms, which will be the last lecture in this module on machine learning.